

Лабораторная работа №3

Коляды Екатерины z3342

13 мая 2026 г.

1 Цель работы

Познакомиться с основными алгоритмическими конструкциями, используемыми при создании CLI-программ. Продемонстрировать эффективность механизма рекурсии на классической задаче, реализовать хранение результатов в динамической структуре данных и освоить базовый workflow Git с автоматизацией деплоя.

2 CLI-калькулятор вещественных чисел

Разработана консольная программа **CALC**, принимающая три аргумента: два операнда и оператор (**plus**, **minus**, **power**).

- Операции сложения и вычитания реализованы напрямую.
- Возведение в степень выполнено через цикл умножений (без использования `std::pow`).
- Предусмотрена проверка корректности ввода и обработка неизвестных операций.
- Вся логика вычислений вынесена в отдельную функцию, соответствующую требованиям лабораторной.

3 Задача о ханойской башне

Алгоритм переноса n дисков реализован рекурсивно. Каждый шаг перемещения сохраняется в пользовательский двусвязный список на основе класса **Node** с указателями **prev** и **next**.

- Рекурсивная функция вызывает себя для $n - 1$ дисков, фиксирует ход, затем повторяет вызов для оставшихся дисков.
- После завершения рекурсии список выводится на экран с нумерацией шагов.
- Перед завершением программы вся выделенная динамическая память освобождается путём последовательного обхода списка и вызова **delete**.
- Общее количество ходов соответствует формуле $2^n - 1$, что подтверждает оптимальность решения.

```

1  #include <iostream>
2  #include <string>
3  #include <cmath>
4  using namespace std;
5  //g++ main.cpp -o build/main.exe -fexec-charset=GBK -g or ctrl shift B
6  //.\build\main.exe
7
8  int calc() {
9      string operation;
10     double num1, num2;
11
12     cout << "Enter first number:";
13     cin >> num1;
14     cout << "Enter operation:";
15     cin >> operation;
16     cout << "Enter second number:";
17     cin >> num2;
18
19     if (operation == "plus"){
20         cout << num1 + num2;
21     }
22     else if(operation == "minus"){
23         cout << num1 - num2;
24     }
25     else if(operation == "power"){
26         double result = 1.0;
27         int exp = static_cast<int>(num2);
28         for (int i = 0; i < exp; ++i) {
29             result *= num1;
30         }
31         cout << result;
32     }
33     else {
34         cout << "Not a recognize operation";
35     }
36
37     return 0;
38 }
39
40 int main() {
41     calc();
42     return 0;
43 }

```

Рис. 1: Исходный код CLI-калькулятора

```

class Node {
public:
    string move;
    Node* prev; // POINTER: Указатель на предыдущий узел
    Node* next; // POINTER: Указатель на следующий узел

    // Конструктор: при создании узла указатели сразу инициализируются nullptr
    Node(string m) : move(m), prev(nullptr), next(nullptr) {}
};

Node* head = nullptr;
Node* tail = nullptr;
int moveCount = 0;

void addMove(string move) {
    Node* ptr = new Node(move);
    ptr->prev = tail;
    ptr->next = nullptr;
    if (tail != nullptr) {
        tail->next = ptr;
    } else {
        head = ptr;
    }
    tail = ptr;
    moveCount++;
}

void printList() {
    Node* current = head;
    int step = 1;

    // POINTER: Цикл продолжается, пока current не станет nullptr (конец списка)
    while (current != nullptr) {
        // POINTER: Оператор -> разменивает указатель current и обращается к полю move
        cout << "Step " << step++ << ": " << current->move << endl;
        // POINTER: Сдвигаем указатель current по цепочке: присваиваем ему адрес следующего узла
        current = current->next;
    }
    cout << "Total moves: " << moveCount << endl;
}

void freeList() {
    Node* current = head;

    while (current != nullptr) {
        Node* next = current->next;
        delete current;
        current = next;
    }
    head = nullptr;
    tail = nullptr;
    moveCount = 0;
}

void towerofhanoi(int n, int a, int b, int c) {
    if (n == 0) {
        return;
    }
    towerofhanoi(n - 1, a, c, b);
    addMove("Move disk from " + to_string(a) + " to " + to_string(c));
    towerofhanoi(n - 1, b, a, c);
}

int main() {
    int n;
    cout << "Enter number of disks: ";
    cin >> n;

    towerofhanoi(n, 1, 2, 3);

    cout << "\n--- Moves list ---" << endl;
    printList();

    freeList(); // Освобождаем все динамическую память перед выходом

    return 0;
}

```

Рис. 2: Исходный код решения задачи о ханойской башне

4 Заключение

В ходе лабораторной работы реализованы две самостоятельные программы на C++ с использованием циклов, условных операторов и рекурсии. Освоены принципы работы с динамической памятью через двусвязный список и ручное управление временем жизни объектов. Настроен workflow Git с разделением окружений (`dev/stg/prd`) и созданы скрипты для безопасного деплоя и отката изменений.